

AE 8883: Spacecraft Engineering I

Software Verification & Validation(V&V)
October 12, 2022



Ebenezer Arunkumar and Mason Starr

When is software ready for release?

When is software ready for release?

A few criteria:

- When you can argue that it is dependable
- When we have finished testing
- When quality is high

Any others?

Software Quality

- What does high-quality software mean?
- For spacecraft software this could be:
 - How quickly it runs
 - How secure is it
 - How fault tolerant it is
 - How understandable it is
 - How easily it can be transferred to another project
- Quality is hard to measure and assess objectively! Also could be mission/program specific

Quality Attributes

- Describe desired properties of the system
- Developers prioritize attributes and design a system that meets the chosen thresholds
- Dependability
 - Ability to consistently offer correct functionality, even under unforeseen or unsafe conditions
- Performance
 - Can the system meet the timing requirements – especially important for real time systems like spacecraft
- Security
 - Ability to protect information from unauthorized access while providing service to authorized users.

Quality Attributes

- Modifiability
 - Ability to enhance software by fixing issues or adding features,
- Testability
 - Ability to easily identify faults in a system
 - Probability that a fault will result in a visible failure
- Usability
 - Ability to enable users to interface with the system
 - How easy it is for developers to use the system and add new features
- Qualities often conflict!
- Important to decide what is important, and set a threshold on what the project believes to be **good enough**
- Software is shown to meet the thresholds of the quality attributes through **Verification & Validation**

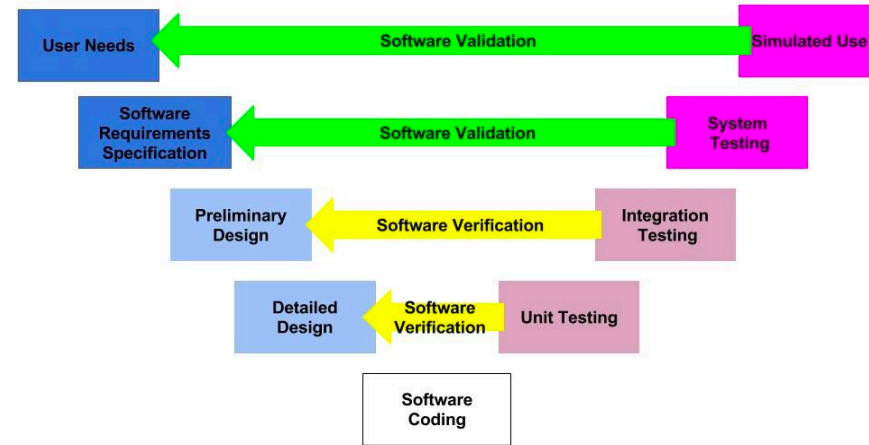
What is V&V?

Verification:

- The process of proving that the software conforms to its specified functional and non-functional requirements.
- "Are we building the product right?"

Validation:

- The process of proving that the software meets the customer's true requirements, needs, and expectations
- "Are we building the right product?"



Software Verification

Is the implementation consistent with its specification?

- Does the software work under conditions we set?
- Usually based on requirements

Multiple things need to be checked against the requirements:

- High-level design and architecture
- Test cases
- Source Code

Essentially, verification is an experiment

Software Validation

- Does the product work in the real world?
 - Does the software fulfill the users' actual needs?

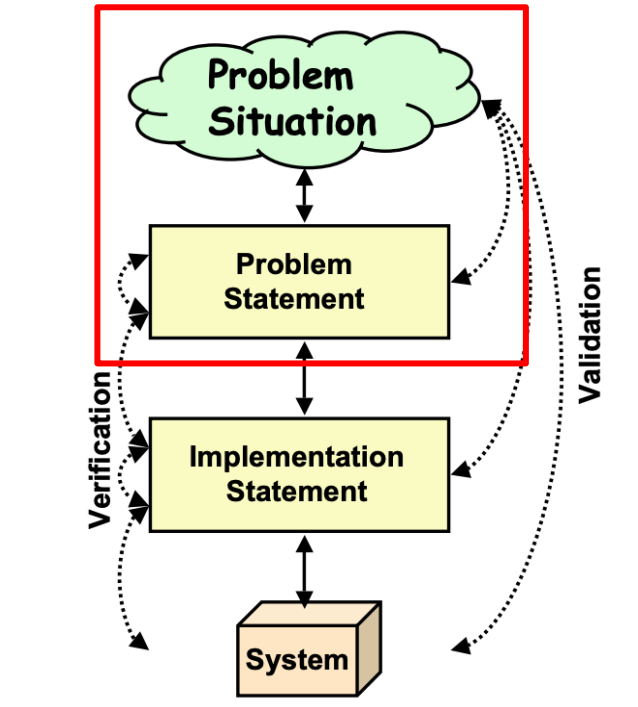
Not the same as conforming to a specification!!

- If we specify two buttons and implement all behaviors related to those buttons, we can achieve verification.
- If the user expected a third button, we have not achieved validation.

Validation is **harder** than verification

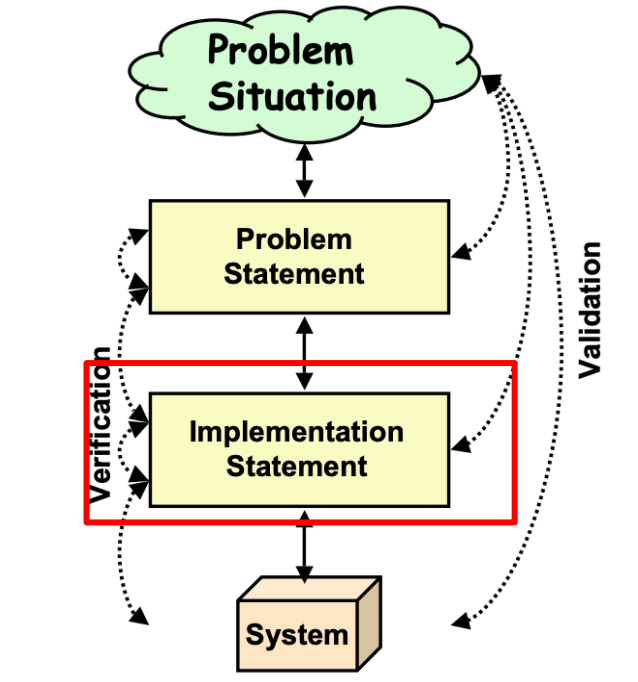
User Needs

- What will this software be used for ?
- What is the purpose of the software?
- Take into account the needs of **all** stakeholders



Software Requirements Specification

- How should the software be architected to meet the needs of the user?
- What design considerations should be taken into account?
- How tolerant should the system be to faults?



Design and Implementation

System Level Preliminary Design

- What should the different pieces of the software system be?
- How do the different pieces fit together?

Subsystem level detailed design

- How should the subsystem achieve its function?
- Does the subsystem stay within its narrow scope?
- Does the subsystem correctly interface with other subsystems according to the system level design?

Implementation

- Does the software correctly implement the system and subsystem level requirements?



Unit & Integration Testing

Unit Testing

- Checks the logic of your code at a function level
- Do your functions do what you expect them to?

Integration Testing

- Checks that your system behaves properly when a subset of subsystems are integrated together

Types of Dynamic Verification

- Testing: Formulating sets of inputs to demonstrate requirement satisfactions
- Fuzzing: Generating semi-random input to locate crashes and other anomalies
- Taint Analysis: Monitoring how faults spread by corrupting system variables



System Level Testing

- Does the full integrated system meet the software requirements?
- Does the full system meet the needs of the users?
- Does the system perform as expected?
- Often done in the loop with user input/hardware
 - HITL/DITL Testbed

How much testing should we do?

- Trade-off between what we want, what we need, and what we've got:
 - **Better, faster, or cheaper – pick any two"**
- Establish criteria for acceptance of the system
 - Finding all faults is nearly impossible
 - How good is "good enough"?
 - Quality is subjective, so set objective thresholds
- V&V does not end with the release of the software:
 - Software is always evolving
 - When code changes, rerun old tests to ensure that the tested elements still work

Learning from V&V

- V&V will give a lot of insight into the robustness of your software system
- Try to learn from mistakes
 - Collect data during development
 - Fault information, bug reports, project metrics(complexity, # classes, # lines of code, test coverage, etc.)
 - Classify faults into categories
 - Look for common mistakes and learn how to avoid them
 - DOCUMENT EVERYTHING!

VISORS Case Study: Hosted Software Application

User Needs:

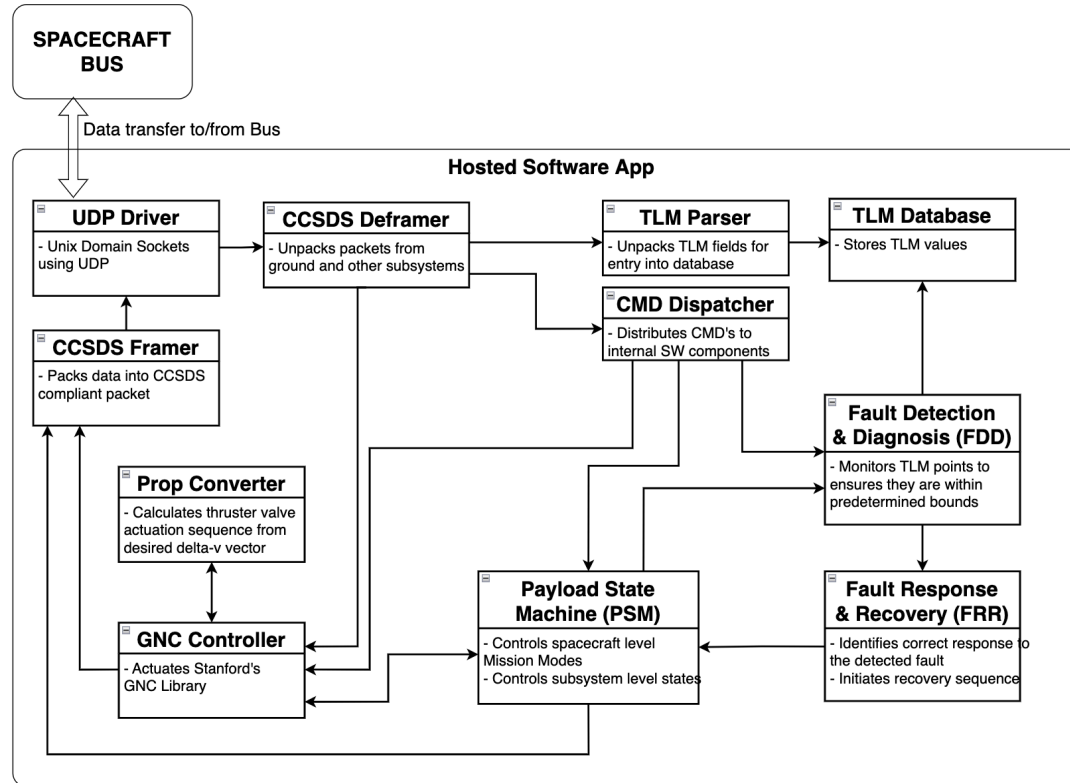
- Software system to control GNC algorithms, run Fault Detection & Recovery logic, and control payload state machine

Software Requirements:

- The HSA shall interface with the other payload subsystems
- The HSA shall continuously monitor telemetry
- The HSA shall control the state of the payload subsystems
- Etc...

VISORS Case Study: Hosted Software Application

Design:



VISORS Case Study: Hosted Software Application

Unit Testing:

```
test 1
  Start 1: SSDL_CCSDSDeframer_ut_exe

1: Test command: /Users/earunkumar/fprime-devel/visors-software/SSDL/s
1: Test timeout computed to be: 10000000
1: [STest::Random] Generated seed 208511 from system time
1: [=====] Running 9 tests from 1 test suite.
1: [-----] Global test environment set-up.
1: [-----] 9 tests from Deframer
1: [ RUN      ] Deframer.TestMoreNeeded
1:
1: *****
1: Send a frame buffer to the mock protocol, expecting more needed
1: *****
1: [      OK ] Deframer.TestMoreNeeded (0 ms)
1: [ RUN      ] Deframer.TestSuccess
1:
1: *****
1: Send a frame buffer to the mock protocol, expecting success
1: *****
1: [      OK ] Deframer.TestSuccess (0 ms)
1: [ RUN      ] Deframer.TestBadChecksum
1:
1: *****
1: Send a frame buffer to the mock protocol, expecting bad checksum
1: *****
1: [      OK ] Deframer.TestBadChecksum (0 ms)
1: [ RUN      ] Deframer.TestBadSize
1:
1: *****
1: Send a frame buffer to the mock protocol, expecting bad size
1: *****
1: [      OK ] Deframer.TestBadSize (0 ms)
1: [ RUN      ] Deframer.TestComInterface
1:
1: *****
```

```
1: *****
1: Route a com packet
1: *****
1: [      OK ] Deframer.TestComInterface (0 ms)
1: [ RUN      ] Deframer.TestFileInterface
1:
1: *****
1: Route a file packet
1: *****
1: [      OK ] Deframer.TestFileInterface (0 ms)
1: [ RUN      ] Deframer.TestUnknownInterface
1:
1: *****
1: Attempt to route a packet of unknown type
1: *****
1: [      OK ] Deframer.TestUnknownInterface (0 ms)
1: [ RUN      ] Deframer.TestCCSDSInterface
1:
1: *****
1: Attempt to route a CCSDS packet
1: *****
1: [      OK ] Deframer.TestCCSDSInterface (0 ms)
1: [ RUN      ] Deframer.TestBufferOutUnconnected
1:
1: *****
1: Test routing a file packet when bufferOut is unconnected
1: *****
1: [      OK ] Deframer.TestBufferOutUnconnected (0 ms)
1: [-----] 9 tests from Deframer (4 ms total)
1:
1: [-----] Global test environment tear-down
1: [=====] 9 tests from 1 test suite ran. (4 ms total)
1: [ PASSED ] 9 tests.
1/1 Test #1: SSDL_CCSDSDeframer_ut_exe ..... Passed    0.67 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) = 0.69 sec
[100%] Built target SSDL_CCSDSDeframer_check
```

VISORS Case Study: Hosted Software Application

Integration Testing:

```
~/fprime-devel/visors-software/SSDL/support/CCSDSFramingTest ➤ devel ? @1
$ build-fprime-automatic-native/bin/Darwin/SSDL_support_CCSDSFramingTest
Hit Ctrl-C to quit
Sent vs Recv: #1 vs # 1: TEST SUCCESSFUL!
Sent vs Recv: #2 vs # 2: TEST SUCCESSFUL!
Sent vs Recv: #3 vs # 3: TEST SUCCESSFUL!
Sent vs Recv: #4 vs # 4: TEST SUCCESSFUL!
Sent vs Recv: #5 vs # 5: TEST SUCCESSFUL!
Sent vs Recv: #6 vs # 6: TEST SUCCESSFUL!
^CWaiting for threads...
Exiting...
```

System Testing: In progress!

- Testing HSA on flight hardware integrated with the BCT subsystem

LF Case Study: Mission Operations Software

User = Lunar Flashlight Operators

- Variety of skill and experience levels, particularly on the command line

User needs

- Quickly generate products and files
- Interface with AMPCS to work around certain hurdles
- Reduce opportunity for error
- Save time during tactical and strategic operations
- **Allow less experienced operators to succeed**

```
[CSH cubesat@lfops1 ~ ] alias go_dsn  
chill --spacecraftID 875 --venueType TESTSET --bufferedInput FSW --inputFormat LEOT_TF  
--downlinkConnectionType SERVER_SOCKET -M DSN_contact --fswUplinkPort 8899 --autoRun
```

LF Case Study: Downlink Helper

Problem: Downlinking recorded telemetry takes way too much time

AMPCS FSW Uplink Control: cubesat_session (430)

File Window Help

Venue: TESTSET Mission: LUNAR_FLASHLIGHT Spacecraft ID: 875

Immediate Command Command Builder Send File Load Send SCMF Send Raw Data File Fault Injection Wizard

Search By: Stem

DWN

FILE_DWN_CANCEL
FILE_DWN_FORCE_REMOVE_APID
FILE_DWN_QUERY_APID
FILE_DWN_QUERY_APID_POR_CNT
FILE_DWN_REMOVE_APID
FILE_DWN_REMOVE_APID_ILLEGALS
FILE_DWN_REMOVE_APID_POR_CNT
FILE_DWN_SEND_APID
FILE_DWN_SEND_APID_POR_CNT
FILE_DWN_SEND_FILE
FILE_DWN_SEND_PARTIAL
FILE_DWN_SET_PACKET_TIMEOUT

Command Information

Stem: FILE_DWN_SEND_APID_POR_CNT
Opcode (16 bits): 0xe209
Module:
Description: Search files with the given APID created for given hard power on reset (POR) count value within the given timerange. Divide it into packets and send the packets for transmission

Command Arguments

apid (U32): 198 0 to 4294967295
porCnt (U32): 172 0 to 4294967295
startTime (U32): 073046212 0 to 4294967295
endTime (U32): 073047559 0 to 4294967295
queryTimeout (U32): 0 0 to 4294967295

Preview Validate Clear

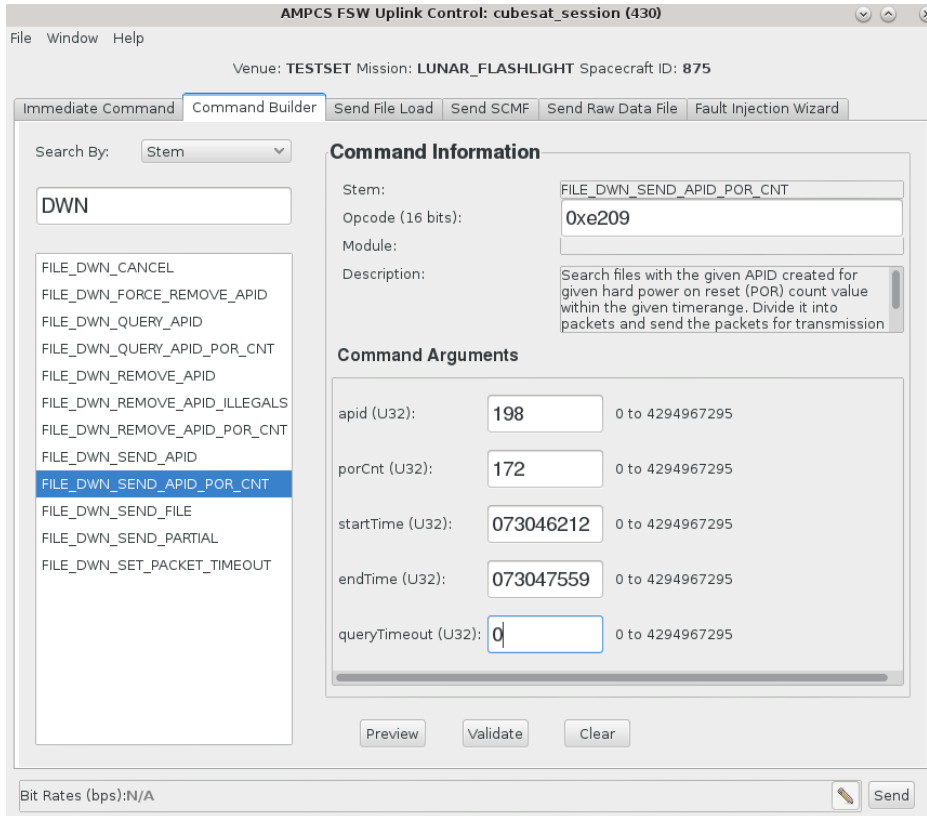
Bit Rates (bps):N/A Send

User needs:

- **Less typing to reduce time and errors**
- Automatic tracking of what has been downlinked and what hasn't
- Automatic management of time ranges
- Don't put the burden of remembering and working with these numbers on the operators

LF Case Study: Downlink Helper

Problem: Downlinking recorded telemetry takes way too much time



Software requirements:

- The software shall automatically generate and manage SCLK time ranges
- The software shall be easy to use
- The software shall provide a visual representation of downlinked telemetry
- The software shall save its progress in event of a crash or anomaly
- ...

LF Case Study: Downlink Helper

```
[CSH cubesat@lfops1 ~/vm_shared/ORT/ORT2/Scripts ] python apids-1by1.py --help
Script for querying APIDs
Usage: query_apids.py <Mode> <AMPCS session ID> <start bookend SCLK> <end bookend SCLK>
Valid modes: downlink query test
```

Iteration 1: Command line script

- Developed under a very tight timeline

Verification

- Satisfied only some of the users' needs
 - Still required manual typing and reference of other documents for times
- Not enough time to pass a full verification process

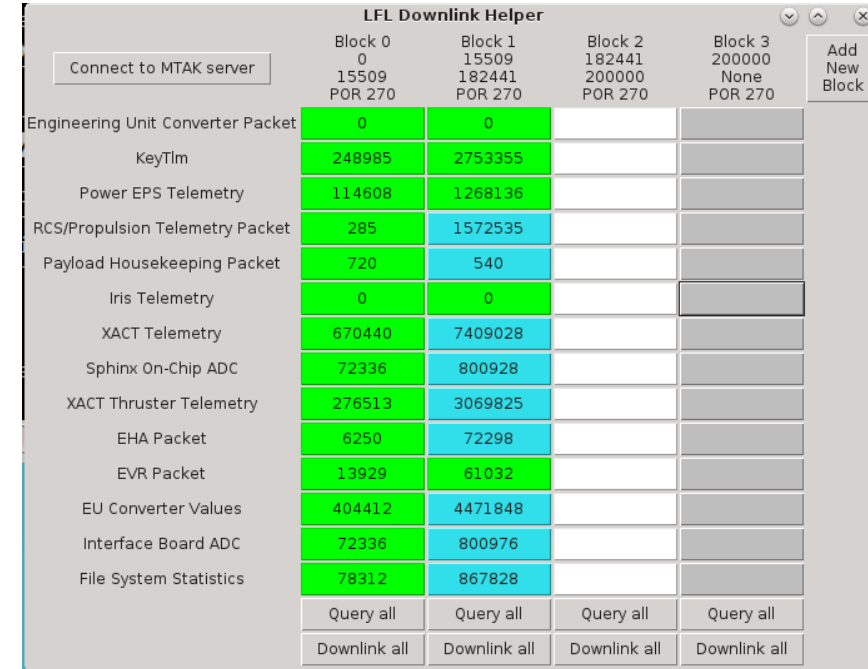
Validation

- Script was used during ORTs
- Script was very helpful for experienced operators, not so much for newer ones
- Usage, notes, and feedback were collected in as-run procedures
- Overall, the script was helpful, but not enough

LF Case Study: Downlink Helper

Iteration 2: Integrated GUI

- Far more development time
- Design: Sketched out concept on paper using feedback from old and new operators
- Subsystem design and implementation: Using Python object oriented approach informed by personal experience
- Unit testing: Testing individual Python classes
- System testing performed on LF testbed
- Testing was performed **cheaper** and **faster**, not the most robust



The screenshot shows a window titled "LFL Downlink Helper". It contains a table with 4 columns representing different data blocks and a list of telemetry packets on the left. The first column has a "Connect to MTAK server" button. The data is as follows:

	Block 0 0 15509 POR 270	Block 1 15509 182441 POR 270	Block 2 182441 200000 POR 270	Block 3 200000 None POR 270	Add New Block
Engineering Unit Converter Packet	0	0			
KeyTlm	248985	2753355			
Power EPS Telemetry	114608	1268136			
RCS/Propulsion Telemetry Packet	285	1572535			
Payload Housekeeping Packet	720	540			
Iris Telemetry	0	0			
XACT Telemetry	670440	7409028			
Sphinx On-Chip ADC	72336	800928			
XACT Thruster Telemetry	276513	3069825			
EHA Packet	6250	72298			
EVR Packet	13929	61032			
EU Converter Values	404412	4471848			
Interface Board ADC	72336	800976			
File System Statistics	78312	867828			
	Query all	Query all	Query all	Query all	
	Downlink all	Downlink all	Downlink all	Downlink all	

LF Case Study: Downlink Helper

Verification

- Satisfies initial user needs
- Satisfies all software requirements

Validation

- Has been used successfully during ORTs
- User needs have been revealed that were not in original list
- Documentation in the form of as run procedures and automatically generated log files
- Will need to be revised in the future!

LFL Downlink Helper					
	Block 0 0 15509 POR 270	Block 1 15509 182441 POR 270	Block 2 182441 200000 POR 270	Block 3 200000 None POR 270	Add New Block
Engineering Unit Converter Packet	0	0			
KeyTlm	248985	2753355			
Power EPS Telemetry	114608	1268136			
RCS/Propulsion Telemetry Packet	285	1572535			
Payload Housekeeping Packet	720	540			
Iris Telemetry	0	0			
XACT Telemetry	670440	7409028			
Sphinx On-Chip ADC	72336	800928			
XACT Thruster Telemetry	276513	3069825			
EHA Packet	6250	72298			
EVR Packet	13929	61032			
EU Converter Values	404412	4471848			
Interface Board ADC	72336	800976			
File System Statistics	78312	867828			
	Query all	Query all	Query all	Query all	
	Downlink all	Downlink all	Downlink all	Downlink all	